JavaScript Data Types

A Comprehensive Reference Guide

Introduction

In JavaScript, Data Types define the type of value a variable can store. JavaScript is a dynamically typed language, which means you don't need to specify the data type explicitly.

The most common data types are:

- String
- Number
- Boolean
- Object
- Array

1. String

A string is used to store text or characters. Strings are written inside:

- " " (double quotes)
- ' ' (single quotes)
- ` ` (backticks)

Example:

```
let name = "Mayur";
let city = 'Ahmedabad';
let message = `Hello World`;

console.log(name);
console.log(city);
console.log(message);
```

Output:

```
Mayur
Ahmedabad
Hello World
```

2. Number

The number data type is used for numeric values. It includes:

- Integers
- Decimals

Example:

```
let age = 25;
let price = 99.99;
```

```
console.log(age);
console.log(price);
```

Output:

```
25
99.99
```

3. Boolean

A boolean represents true or false values. Used in conditions and comparisons.

Example:

```
let isLoggedIn = true;
let isAdmin = false;
```

```
console.log(isLoggedIn);
console.log(isAdmin);
```

Output:

```
true
false
```

4. Object

An object stores multiple values using key-value pairs.

Syntax:

```
let objectName = {
  key: value
};
```

Example:

```
let person = {
   name: "Mayur",
   age: 28,
   city: "Ahmedabad"
};

console.log(person.name);
console.log(person.age);
```

Output:

```
Mayur
28
```

5. Array

An array stores multiple values in a single variable. Arrays use square brackets [ ].

Example:

```
let fruits = ["Apple", "Banana", "Mango"];

console.log(fruits);
```

Output:

```
["Apple", "Banana", "Mango"]
```

Template Literals in JavaScript

ES6 Feature — Backtick Strings & Expressions

Introduction

Template literals are a way to create strings with embedded expressions using backticks (`) instead of single (' ') or double (" ") quotes.

They were introduced in ES6 (ECMAScript 2015).

Template literals make it easier to:

- Insert variables into strings
- Write multi-line strings
- Format text cleanly

1. Basic Syntax

Template literals use backticks.

Example:

```
let name = "Mayur";


console.log(`Hello ${name}`);
```

Output:

```
Hello Mayur
```

2. Without Template Literals (Old Method)

Before ES6 we used string concatenation.

Example:

```
let name = "Mayur";
```

```
console.log("Hello " + name);
```

Output:

```
Hello Mayur
```

3. Using Multiple Variables

Example:

```
let firstName = "Mayur";
let lastName = "Patel";


console.log(`My name is ${firstName} ${lastName}`);
```

Output:

```
My name is Mayur Patel
```

4. Expressions Inside Template Literals

Example:

```
let a = 10;
let b = 5;


console.log(`Sum is ${a + b}`);
```

Output:

```
Sum is 15
```

5. Multi-line Strings

Template literals allow multi-line strings easily.

Example:

```
let message = `Hello
Welcome to
JavaScript`;
```

```
console.log(message);
```

Output:

```
Hello
Welcome to
JavaScript
```

Destructuring in JavaScript

ES6 Feature — Unpack Arrays & Objects Easily

Introduction

Destructuring is a JavaScript feature that allows you to extract values from arrays or objects and assign them to variables easily.

It was introduced in ES6 (ECMAScript 2015). Instead of accessing values one by one, destructuring lets you unpack them quickly.

1. Array Destructuring

Used to extract values from an array.

Without Destructuring:

```
let fruits = ["Apple", "Banana", "Mango"];


let first = fruits[0];
let second = fruits[1];
let third = fruits[2];

console.log(first);
console.log(second);
console.log(third);
```

With Destructuring:

```
let fruits = ["Apple", "Banana", "Mango"];


let [first, second, third] = fruits;

console.log(first);
console.log(second);
console.log(third);
```

Output:

```
Apple
Banana
Mango
```

2. Skipping Values in Array Destructuring

You can skip elements using commas.

Example:

```
let fruits = ["Apple", "Banana", "Mango"];

let [first, , third] = fruits;

console.log(first);
console.log(third);
```

Output:

```
Apple
Mango
```

3. Object Destructuring

Used to extract values from an object.

Example Object:

```
let person = {
  name: "Mayur",
  age: 28,
  city: "Ahmedabad"
};
```

Without Destructuring:

```
let name = person.name;
let age = person.age;
```

```
let city = person.city;
```

With Destructuring:

```
let {name, age, city} = person;

console.log(name);
console.log(age);
console.log(city);
```

Output:

```
Mayur
28
Ahmedabad
```

4. Renaming Variables

You can rename variables while destructuring.

Example:

```
let person = {
  name: "Mayur",
  age: 28
};

let {name: userName, age: userAge} = person;


console.log(userName);
console.log(userAge);
```

Output:

```
Mayur
28
```

5. Default Values

You can assign default values if a property doesn't exist.

Example:

```
let person = {
  name: "Mayur"
};

let {name, age = 25} = person;

console.log(name);
console.log(age);
```

Output:

```
Mayur
25
```

6. Destructuring in Functions

Very commonly used when passing objects to functions.

Example:

```
function displayUser({name, age}) {
  console.log(name);
  console.log(age);
}

let user = {
  name: "Rahul",
  age: 30
};

displayUser(user);
```

Output:

```
Rahul
30
```

JavaScript Array Methods

map • filter • reduce • find

.map() – Transform Each Element

- Used when you want to change every element in an array
- Returns a new array with modified values

Basic Syntax:

```
array.map(function(element, index, array) {
  return newValue;
});
```

Arrow Function Syntax:

```
array.map((element, index) => {
  return newValue;
});
```

Normal Function – Multiply each number by 2

Example:

```
let numbers = [1, 2, 3, 4];

let result = numbers.map(function(num) {
    return num * 2;
});

console.log(result);
```

Output:

```
[2, 4, 6, 8]
```

Arrow Function – Multiply each number by 2

Example:

```
let numbers = [1, 2, 3, 4];

let result = numbers.map(num => num * 2);

console.log(result);
```

Output:

```
[2, 4, 6, 8]
```

.filter() – Select Elements Based on Condition

- Used when you want to keep only elements that match a condition
- Returns a new array with filtered values

Basic Syntax:

```
array.filter(function(element, index, array) {
  return condition;
});
```

Normal Function – Get numbers greater than 10

Example:

```
let numbers = [5, 10, 15, 20];

let result = numbers.filter(function(num) {
    return num > 10;
});

console.log(result);
```

Output:

```
[15, 20]
```

Arrow Function – Get numbers greater than 10

Example:

```
let numbers = [5, 10, 15, 20];
```

```
let result = numbers.filter(num => num > 10);

console.log(result);
```

Output:

```
[15, 20]
```

.reduce() – Convert Array to Single Value

- Used when you want to combine all elements into one value (e.g. sum, total, count)

Example – Sum of Numbers

```
let numbers = [1, 2, 3, 4];

let total = numbers.reduce(function(sum, num) {
    return sum + num;
}, 0);

console.log(total);
```

Output:

```
10
```

.find() – Return First Matching Element

.find() is an array method used to return the first element that satisfies a condition. It stops searching once the first match is found.

Basic Syntax:

```
array.find(function(element, index, array) {
  return condition;
});
```

Example 1 – Simple Number Example

```
let numbers = [5, 10, 15, 20];

let result = numbers.find(function(num) {
    return num > 10;
});

console.log(result);
```

Output:

```
15
```

Example 2 – Using Arrow Function

```
let numbers = [5, 10, 15, 20];

let result = numbers.find(num => num > 10);

console.log(result);
```

Output:

```
15
```

Async/Await in JavaScript

Handle Asynchronous Operations Cleanly

Introduction

- Async/Await is used to handle asynchronous operations in a cleaner and more readable way
- It is built on top of Promises
- Makes async code look like synchronous code

1. async Keyword

- Used before a function
- It makes the function always return a Promise

Example:

```
async function greet() {
    return "Hello";
}


greet().then(result => console.log(result));
```

Output:

```
Hello
```

2. await Keyword

- Used inside async function only
- It waits for the Promise to resolve

Example:

```
function fetchData() {
    return new Promise((resolve) => {
        setTimeout(() => {
```

```
            resolve("Data received");
        }, 2000);
    });
}

async function getData() {
    let result = await fetchData();
    console.log(result);
}

getData();
```

Output (after 2 sec):

```
Data received
```

Real QA Example

A practical example showing how Async/Await is used in a login test scenario:

Example:

```
async function loginTest() {
    let response = await fetch("/login");

    if (response.status === 200) {
        console.log("Login successful");
    } else {
        console.log("Login failed");
    }
}
```

Async/Await in JavaScript

Handle Asynchronous Operations Cleanly

Introduction

- Async/Await is used to handle asynchronous operations in a cleaner and more readable way
- It is built on top of Promises
- Makes async code look like synchronous code

1. async Keyword

- Used before a function
- It makes the function always return a Promise

Example:

```
async function greet() {
    return "Hello";
}


greet().then(result => console.log(result));
```

Output:

```
Hello
```

2. await Keyword

- Used inside async function only
- It waits for the Promise to resolve

Example:

```
function fetchData() {
    return new Promise((resolve) => {
        setTimeout(() => {
```

```
            resolve("Data received");
        }, 2000);
    });
}

async function getData() {
    let result = await fetchData();
    console.log(result);
}

getData();
```

Output (after 2 sec):

```
Data received
```

Real QA Example

```
async function loginTest() {
    let response = await fetch("/login");

    if (response.status === 200) {
        console.log("Login successful");
    } else {
        console.log("Login failed");
    }
}
```

Promise in JavaScript

Handle Async Results — Success or Failure

Introduction

A Promise is an object that represents the result of an asynchronous operation (something that takes time).

- It gives a result in the future
- Either success or failure

Why Promises?

Used for operations like:

- API calls
- Database requests
- File reading
- Timers (setTimeout)

Without promises → messy callbacks ✗
With promises → clean & manageable code ✓

Promise States

A Promise has 3 states:

State	Meaning
Pending	Still running
Fulfilled	Completed successfully
Rejected	Failed

Basic Syntax

```
let promise = new Promise(function(resolve, reject) {

    // async task

    if (true) {
        resolve("Success");
    } else {
        reject("Error");
    }

});
```

Example 1 – Simple Promise

```
let myPromise = new Promise((resolve, reject) => {

    let success = true;

    if (success) {
        resolve("Task completed");
    } else {
        reject("Task failed");
    }

});

myPromise
.then(result => console.log(result))
.catch(error => console.log(error));
```

Output:

```
Task completed
```

.then() and .catch()

Method	Purpose
`.then()`	Runs when promise is fulfilled
`.catch()`	Runs when promise is rejected

Example 2 – Using setTimeout

```
function fetchData() {
    return new Promise((resolve, reject) => {

        setTimeout(() => {
            resolve("Data received");
        }, 2000);

    });
}

fetchData()
.then(data => console.log(data))
.catch(err => console.log(err));
```

Output (after 2 seconds):

```
Data received
```

Promise Chaining

You can chain multiple .then() calls.

```
let promise = new Promise((resolve) => {
    resolve(10);
});

promise
.then(num => num * 2)
.then(num => num + 5)
.then(result => console.log(result));
```

Output:

```
25
```

Real Example (API)

```
fetch("https://jsonplaceholder.typicode.com/users")
.then(response => response.json())
.then(data => console.log(data))
.catch(error => console.log(error));
```

JavaScript Objects

Store & Manage Collections of Values

Introduction

- In JavaScript an object is a variable that can hold multiple values
- It is a location where a collection of values is stored
- Objects are among the most basic data types in JavaScript
- Every object contains properties and types, which are a single unit
- Objects are not primitive, unlike primitive data types

1. Using Object Literal

Syntax:

```
let objectName = {
   key: value
};
```

Example:

```
let person = {
    name: "Mayur",
    age: 28,
    city: "Ahmedabad"
};

console.log(person.name);
```

Output:

```
Mayur
```

2. Using new Object()

- Create object using JavaScript built-in constructor

Syntax:

```
let obj = new Object();
```

Example:

```
let person = new Object();

person.name = "Mayur";
person.age = 28;

console.log(person);
```

Output:

```
{ name: "Mayur", age: 28 }
```

3. Using Constructor Function (Reusable)

- Used when you want to create multiple objects with same structure

Syntax:

```
function ConstructorName(param1, param2) {
    this.param1 = param1;
    this.param2 = param2;
}
```

Example:

```
function Person(name, age) {
    this.name = name;
    this.age = age;
}

let user1 = new Person("Mayur", 28);
let user2 = new Person("Rahul", 30);

console.log(user1);
console.log(user2);
```

Output:

```
{ name: "Mayur", age: 28 }
{ name: "Rahul", age: 30 }
```

Object Creation Methods — Comparison

Method	Syntax	Use Case
Object Literal	{}	Most common, simple
new Object()	new Object()	Rarely used
Constructor Function	new FunctionName()	Reusable objects

Advanced Example — Nested Objects & Arrays

```
let testing = [
    {
        name: 'Rahul',
        role: 'qa',
        details: {
            var1: 'ccv',
            var2: 'bvbb'
        }
    },
    {
        name: 'Mayur',
        role: 'devops',
        special: [
            { skill: 'AWS' },
            { skill: 'Docker' }
        ]
    }
];

console.log(testing[0].name);
// Output: Rahul
```

```
console.log(testing[0].details.var1);
// Output: ccv


console.log(testing[1].special[1].skill);
// Output: Docker
```

Output:

```
Rahul
```

```
ccv
Docker
```

JavaScript Classes

Class Declarations & Class Expressions

Introduction

- In JavaScript, classes are a special type of functions
- We can define the class just like function declarations and function expressions
- The JavaScript class contains various class members within a body including methods or constructor
- The class is executed in strict mode — code containing silent errors throws an error

The class syntax contains two components:

- Class declarations
- Class expressions

Class Declarations

- A class can be defined by using a class declaration
- A class keyword is used to declare a class with any particular name
- According to JavaScript naming conventions, the name of the class always starts with an uppercase letter

Syntax:

```
class Person {
    constructor(name, role) {
        this.name = name;
        this.role = role;
    }

    display() {
        console.log(this.name + " is a " + this.role);
    }
}
```

Example (How to use):

```
let p1 = new Person("Rahul", "QA");
p1.display();
```

Output:

```
Rahul is a QA
```

Class Expression

A class expression is when a class is assigned to a variable.

```
const MyClass = class {
    // class body
};
```

1. Unnamed (Anonymous) Class Expression

Syntax:

```
const Person = class {
    constructor(name) {
        this.name = name;
    }

    display() {
        console.log(this.name);
    }
};
```

Usage:

```
let p1 = new Person("Rahul");
p1.display();
```

Key Points:

- Class has no internal name
- We use variable name (Person) to access it
- Most commonly used in real projects

2. Named Class Expression

Syntax:

```
const Person = class Employee {
    constructor(name) {
        this.name = name;
    }

    display() {
        console.log(this.name);
    }
};
```

Usage:

```
let p2 = new Person("Mayur");
p2.display();
```

Points:

- Employee name is only available inside the class
- Outside → only Person works

Internal Use Example:

```
const Person = class Employee {
    constructor(name) {
        this.name = name;
    }

    show() {
        console.log(Employee.name); // works
    }
};

let obj = new Person("Rahul");
obj.show();
```

Unnamed vs Named Class Expression — Comparison

Feature	Unnamed Class Expression	Named Class Expression
Class name	❌No	✅Yes (internal only)
External access	Variable name	Variable name
Internal reference	❌Not possible	✅Possible
Debugging	Harder	Easier (name appears in stack)

JavaScript Prototype Object

Prototype-Based Inheritance & Method Sharing

Introduction

- A prototype is an object from which other objects inherit properties and methods
- JavaScript is a prototype-based language that facilitates objects to acquire properties and features from one another
- Each object contains a prototype object
- Whenever a function is created, the prototype property is added to that function automatically
- This property is a prototype object that holds a constructor property

Syntax:

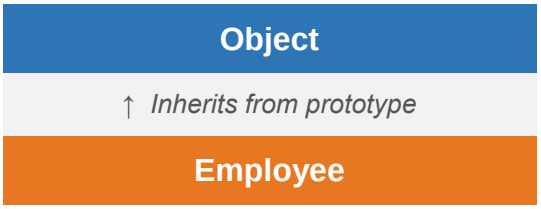
```
ClassName.prototype.methodName
```

Requirement of a Prototype Object

- Whenever an object is created in JavaScript, its corresponding functions are loaded into memory — a new copy of the function is created on each object creation
- In a prototype-based approach, all objects share the same function
- This removes the need for creating a new copy of the function for each object — functions are loaded once into memory

Prototype Chaining

In JavaScript, each object contains a prototype object that acquires properties and methods from it. An object's prototype object may itself contain a prototype object that also acquires properties and methods, and so on. This is known as prototype chaining.



↑ *Inherits from prototype*

employee1

Prototype is Used For

- To share methods between objects instead of duplicating them

Without Prototype (Bad Way)

```
function Person(name) {
    this.name = name;

    this.sayHello = function () {
        console.log("Hello " + this.name);
    };
}

let p1 = new Person("Rahul");
let p2 = new Person("Mayur");

p1.sayHello();
```

Output:

```
Hello Rahul
```

 Each object gets its own copy of sayHello() — wastes memory.

With Prototype (Correct Way)

```
function Person(name) {
    this.name = name;
}

// Adding method to prototype
Person.prototype.sayHello = function () {
    console.log("Hello " + this.name);
};
```

```
let p1 = new Person("Rahul");
let p2 = new Person("Mayur");

p1.sayHello();
p2.sayHello();
```

Output:

```
Hello Rahul
Hello Mayur
```

✅Both objects share the same sayHello() method — saves memory.

JavaScript Constructor Method

Initialize & Create Objects with constructor()

Introduction

A JavaScript constructor method is a special type of method which is used to initialize and create an object. It is called when memory is allocated for an object.

Key Points

- The constructor keyword is used to declare a constructor method
- The class can contain one constructor method only
- JavaScript allows us to use parent class constructor through the super keyword
- If no constructor method is specified, JavaScript uses a default constructor method

Constructor Method Example

```
class Employee {
  constructor() {
    this.id = 101;
    this.name = "Martin Roy";
  }
}

var emp = new Employee();
console.log(emp.id);
```

Output:

```
101
```

Constructor Method Example — super Keyword

The super keyword is used to call the parent class constructor.

```
class CompanyName {
    constructor(company) {
        this.company = 'Testing group';
    }
}

class Employee extends CompanyName {
    constructor(id, name) {
        super();
        this.id = 101;
        this.name = "Martin Roy";
    }
}

var emp = new Employee();
console.log(emp.company);
```

Output:

```
Testing group
```

— End of JavaScript Data Types Reference —